

MATH3900: Optimization for AI and DA

Introduction

Prof. Lexiao Lai
Department of Mathematics
The University of Hong Kong

Semester 1, 2026–2027



► **Weekly meetings:**

	Time	Venue
Lecture	Tue 09:00–09:50	KK101
	Thu 10:00–11:50	KK101
Tutorial	Fri 12:00–12:50	LE2

- **Assessment methods:** 4 assignments (10%), 2 midterm tests (40%), and a final examination (50%).
- **Test dates (tentative):** Test 1: October 6, Test 2: November 17.
- **Assignments:** Submit through Gradescope; deadlines and instructions will be announced on Moodle.
- **Course information and support:** Materials, announcements, test dates, office hours, and teaching-assistant details will be posted on [Moodle](#).
- Use the Moodle discussion forum for course-related questions.
- Any use of artificial intelligence (AI) tools must be acknowledged.
- **Academic integrity: submit your own work.**

Regarding course content:

- ▶ MATH3904 (*Introduction to Optimization*) covers content similar to that of MATH3900, so the two courses are mutually exclusive.
- ▶ The main difference is that MATH3900 focuses on optimization theory and algorithms motivated by applications in AI and data analytics, whereas MATH3904 develops a general theory of optimization.

Regarding degree and major requirements:

- ▶ Both MATH3900 and MATH3904 can be used to satisfy the major requirements for [Applied AI](#) and [Decision Analytics](#).
- ▶ MATH3900 is **not** a disciplinary core or elective course for **Mathematics** majors or minors.
- ▶ Students in the **Mathematics (Intensive)** major should **not** take MATH3900 because MATH3904 is compulsory for that major.

What is an optimization problem?

- ▶ An optimization problem asks us to choose a decision variable x that makes an objective as small as possible while satisfying the constraints:

$$\min_{x \in C} f(x).$$

- ▶ **Decision variable** $x \in \mathbb{R}^d$; **Objective function** $f : \mathbb{R}^d \rightarrow \mathbb{R}$.
- ▶ **Feasible region**: $C \subset \mathbb{R}^d$
- ▶ $x \in C$ is **feasible** and $x \notin C$ is **infeasible**.
- ▶ If $C = \mathbb{R}^d$, the problem is **unconstrained**; otherwise it is **constrained**.
- ▶ How about maximization problems?
- ▶ A maximization problem can be written as a minimization problem:

$$\max_{x \in C} f(x) \quad \Longleftrightarrow \quad \min_{x \in C} [-f(x)].$$

Consider the optimization problem:

$$\min_{x \in C} f(x).$$

Definition

For $f : C \rightarrow \mathbb{R}$ and $x^* \in C$:

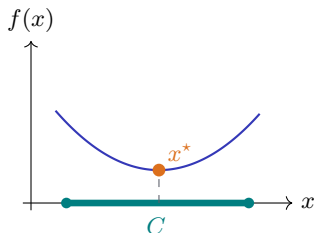
- ▶ x^* is a **global minimizer** if $f(x^*) \leq f(x)$ for every $x \in C$.
- ▶ x^* is a **local minimizer** if there exists $\varepsilon > 0$ such that $f(x^*) \leq f(x)$ for all $x \in C$ with Euclidean distance $\|x - x^*\| \leq \varepsilon$.
- ▶ The notation $\arg \min_{x \in C} f(x)$ denotes the set of global minimizers.
- ▶ The **optimal value** is $f^* := \inf_{x \in C} f(x)$.
- ▶ If x^* is a global minimizer, then $f(x^*) = f^*$.
- ▶ Is the optimal value always obtained at some $x \in C$?

When does a minimizer exist?

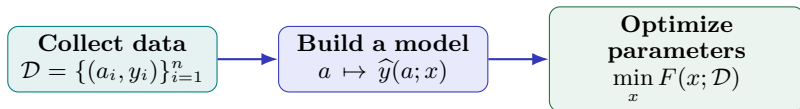
Theorem: Weierstrass theorem

If $C \subset \mathbb{R}^d$ is nonempty and compact and $f : C \rightarrow \mathbb{R}$ is continuous, then f has at least one global minimizer on C .

- ▶ In $\mathbb{R}^d$, a set is compact exactly when it is closed and bounded.
- ▶ Closedness: The set contains its boundary points.
- ▶ Boundedness: The set is contained in a ball.
- ▶ Are these conditions necessary?



- ▶ Many AI and data-science tasks can be formulated as optimization problems.
- ▶ A typical workflow:



Task	Optimization problem
Linear regression	$\min_x (2n)^{-1} \sum_i (a_i^T x - y_i)^2$
Binary classification	$\min_x n^{-1} \sum_i \log(1 + e^{-y_i a_i^T x})$
Hard-margin SVM	$\min_x \frac{1}{2} \ x\ ^2$ subject to $y_i a_i^T x \geq 1$ for all i
Sparse prediction	$\min_x f(x) + \lambda \ x\ _1$
Neural-network training	$\min_x n^{-1} \sum_i \mathcal{L}(h(a_i; x), y_i)$

Example: Linear Regression

- ▶ **Task:** predict a numerical response $y \in \mathbb{R}$ from features $a \in \mathbb{R}^d$.
- ▶ We are given training data $\mathcal{D} = \{(a_i, y_i)\}_{i=1}^n$, where $a_i \in \mathbb{R}^d$ contains the features and $y_i \in \mathbb{R}$ is the observed response.
- ▶ A parameter vector $x \in \mathbb{R}^d$ gives the prediction

$$\hat{y} = a^T x.$$

- ▶ For training sample i , $\hat{y}_i = a_i^T x$, so $a_i^T x - y_i = \hat{y}_i - y_i$ is its prediction error.
- ▶ We choose x by minimizing the mean squared error (MSE):

$$\min_{x \in \mathbb{R}^d} \frac{1}{2n} \sum_{i=1}^n (a_i^T x - y_i)^2.$$

- ▶ This optimization problem has a closed-form solution; we will return to it later.

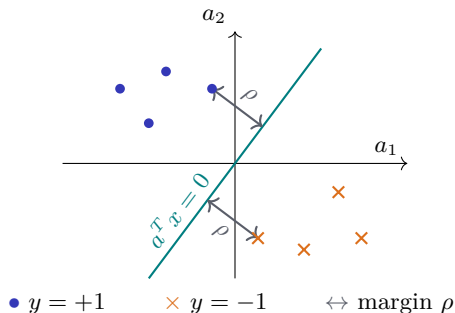
Example: Logistic Regression

- ▶ **Task:** predict a binary class $y \in \{-1, 1\}$ from features $a \in \mathbb{R}^d$.
- ▶ We are given training data $\mathcal{D} = \{(a_i, y_i)\}_{i=1}^n$, where $a_i \in \mathbb{R}^d$ contains the features and $y_i \in \{-1, 1\}$ is the observed class.
- ▶ A parameter vector $x \in \mathbb{R}^d$ assigns the score $a^T x$ to a new observation and predicts $\hat{y} = \text{sign}(a^T x)$.
- ▶ The signed margin $y_i a_i^T x$ is positive for a correct prediction.
- ▶ We choose x by solving

$$\min_{x \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \log(1 + e^{-y_i a_i^T x}).$$

- ▶ This problem does not have a closed-form formula.

Example: Hard-Margin SVM



- **Task:** for linearly separable binary data, find the widest-margin classifier by solving

$$\begin{aligned} \min_{x \in \mathbb{R}^d} \quad & \frac{1}{2} \|x\|^2 \\ \text{s.t.} \quad & y_i a_i^T x \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

- This is a **constrained** optimization problem.

Example: Sparse linear regression with Lasso

- **Task:** fit a linear model using only a small subset of the available features.

$$\min_{x \in \mathbb{R}^d} \frac{1}{2n} \sum_{i=1}^n (a_i^T x - y_i)^2 + \lambda \|x\|_1, \quad \|x\|_1 = \sum_{j=1}^d |x_j|.$$

- The parameter $\lambda \geq 0$ balances data fit and sparsity by encouraging some coefficients x_j to be exactly zero.
- When $\lambda > 0$, the ℓ_1 term makes the objective **nonsmooth** at any point with a zero coefficient.

Example: Neural Network Training

- ▶ **Task:** predict a target y from features $a \in \mathbb{R}^q$.
- ▶ We are given training data $\mathcal{D} = \{(a_i, y_i)\}_{i=1}^n$, where $a_i \in \mathbb{R}^q$ contains the features and y_i is the observed target.
- ▶ A neural network with parameters $x \in \mathbb{R}^d$ predicts $\hat{y}_i = h(a_i; x)$.
- ▶ Examples: multilayer perceptron (MLP), convolutional network (CNN), and transformer.
- ▶ For training sample i , the sample loss composes the network prediction with a loss function:

$$\ell_i(x) = \mathcal{L}(h(a_i; x), y_i).$$

Common choices:

1. MSE $\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$ for regression.
 2. cross-entropy $\mathcal{L}(p, y) = -\log p_y$ for classification.
- ▶ We choose x to minimize the average training loss:

$$\min_x F(x), \quad F(x) = \frac{1}{n} \sum_{i=1}^n \ell_i(x).$$

- ▶ This is a **large-scale** problem.

Example: Neural Network Training

$$\min_x F(x), \quad F(x) = \frac{1}{n} \sum_{i=1}^n \ell_i(x).$$

- ▶ Current AI models are trained on massive datasets with billions of parameters d .

Quantity	Example	Reported scale
Training size n	ImageNet-1K	1.28 million training images
	FineWeb	more than 18.5 trillion tokens
	RedPajama-V2	30.4 trillion estimated tokens
Parameters d	ResNet-50	25.6 million parameters
	DeepSeek-V4-Pro	1.6 trillion parameters
	Kimi K3	2.8 trillion parameters

- ▶ It is difficult to even evaluate F in such large-scale models.
- ▶ Solution: **stochastic optimization**, which uses a small subset (mini-batch) of the data at each time.

Questions we will answer

- ▶ How can an AI or data task be formulated as an optimization problem?
- ▶ What mathematical structure does the problem have?
- ▶ Which algorithm should we use, and why does it work?

▶ **Required material:** the annotated lecture slides.

▶ **Recommended textbooks (optional):**

1. Boyd and Vandenberghe, *Convex Optimization*;
2. Nocedal and Wright, *Numerical Optimization*;
3. Beck, *First-Order Methods in Optimization*.

▶ AI tools can be very helpful when used smartly.

▶ You should always ask not only *what* the answer is, but also *why it is correct*.

Topics and examples

1. **Unconstrained optimization:** linear and logistic regression.
2. **Constrained optimization:** hard-margin SVM.
3. **Nonsmooth optimization:** sparse prediction with ℓ_1 regularization.
4. **Stochastic optimization:** neural-network training with mini-batches.

- **Vectors.** For $u, v \in \mathbb{R}^q$ and $\alpha \in \mathbb{R}$, work coordinate by coordinate: $(u + v)_i = u_i + v_i$ and $(\alpha u)_i = \alpha u_i$. For example,

$$u = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad v = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \quad \implies \quad u + v = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, \quad 2u = \begin{bmatrix} 2 \\ -4 \end{bmatrix}.$$

- **Inner/dot product and Euclidean norm.** Multiply corresponding coordinates and add for the inner product:

$$\langle u, v \rangle = u^T v = \sum_{i=1}^q u_i v_i, \quad \|u\| = \sqrt{\langle u, u \rangle}.$$

For the vectors above, $\langle u, v \rangle = 1 \cdot 3 + (-2) \cdot 4 = -5$ and $\|u\| = \sqrt{5}$.

- **Matrices.** For $M, N \in \mathbb{R}^{p \times q}$, addition and scalar multiplication are also entrywise: $(M + N)_{ij} = M_{ij} + N_{ij}$ and $(\alpha M)_{ij} = \alpha M_{ij}$.

- ▶ **Matrix–vector multiplication.** If $M \in \mathbb{R}^{p \times q}$ and $x \in \mathbb{R}^q$, then $Mx \in \mathbb{R}^p$. Each row of M produces one output entry. For example,

$$M = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad x = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \quad \Rightarrow \quad Mx = \begin{bmatrix} 1 \cdot 3 + 2 \cdot 1 \\ 0 \cdot 3 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}.$$

- ▶ **Matrix–matrix multiplication.** If $N = [w_1 \ \cdots \ w_r] \in \mathbb{R}^{q \times r}$, apply M to each column: $MN = [Mw_1 \ \cdots \ Mw_r] \in \mathbb{R}^{p \times r}$. With

$$N = \begin{bmatrix} 3 & 0 \\ 1 & 2 \end{bmatrix} = [w_1 \ w_2], \quad MN = [Mw_1 \ Mw_2] = \begin{bmatrix} 5 & 4 \\ 1 & 2 \end{bmatrix}.$$

- ▶ When a $p \times q$ matrix multiplies a $q \times r$ matrix, the inner dimensions match and the result has shape $p \times r$.
- ▶ Many practical objectives can be written compactly in matrix–vector form.

Linear Regression: Rewrite the Objective

- Recall the linear-regression objective:

$$\min_{x \in \mathbb{R}^d} \frac{1}{2n} \sum_{i=1}^n (a_i^T x - y_i)^2.$$

- Stack the feature vectors as rows of a matrix and the targets as a vector:

$$A = \begin{bmatrix} a_1^T \\ \vdots \\ a_n^T \end{bmatrix} \in \mathbb{R}^{n \times d}, \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

- Each row of A produces one prediction, so

$$Ax = \begin{bmatrix} a_1^T x \\ \vdots \\ a_n^T x \end{bmatrix} \in \mathbb{R}^n.$$

- ▶ Subtracting the target vector collects all prediction errors:

$$Ax - y = \begin{bmatrix} a_1^T x - y_1 \\ \vdots \\ a_n^T x - y_n \end{bmatrix} \in \mathbb{R}^n.$$

- ▶ The squared Euclidean norm adds the squares of these entries:

$$\|Ax - y\|^2 = \sum_{i=1}^n (a_i^T x - y_i)^2.$$

- ▶ Substitution gives the same problem in matrix form:

$$\min_{x \in \mathbb{R}^d} \frac{1}{2n} \|Ax - y\|^2.$$

Directional Derivative and Gradient

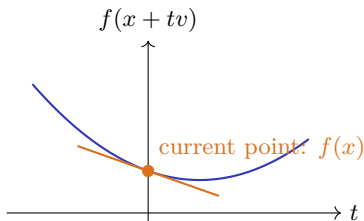
Definition

For $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and $x, v \in \mathbb{R}^d$, the one-sided directional derivative of f at x along v , when it exists, is

$$f'(x; v) = \lim_{t \downarrow 0} \frac{f(x + tv) - f(x)}{t}.$$

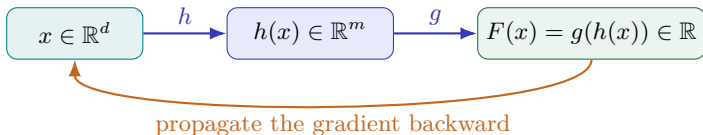
If f is differentiable at x , its gradient is

$$\nabla f(x) = \left(\frac{\partial f(x)}{\partial x_1}, \dots, \frac{\partial f(x)}{\partial x_d} \right)^T, \quad f'(x; v) = \nabla f(x)^T v.$$



The Chain Rule

- Suppose $F(x) = g(h(x))$, with $h = (h_1, \dots, h_m) : \mathbb{R}^d \rightarrow \mathbb{R}^m$ and $g : \mathbb{R}^m \rightarrow \mathbb{R}$.



Theorem

Assume that h is differentiable at x and that g is differentiable at $h(x)$. The Jacobian of the inner map stacks the component gradients as rows:

$$J_h(x) := \begin{bmatrix} \nabla h_1(x)^T \\ \vdots \\ \nabla h_m(x)^T \end{bmatrix} \in \mathbb{R}^{m \times d}.$$

With column gradients, the chain rule can be written in either form:

$$\nabla F(x) = \sum_{j=1}^m [\nabla g(h(x))]_j \nabla h_j(x) = J_h(x)^T \nabla g(h(x)).$$

- Recall $F(x) = (2n)^{-1} \|Ax - y\|^2$. Define the inner and outer maps by

$$h(x) = Ax - y \in \mathbb{R}^n, \quad g : \mathbb{R}^n \rightarrow \mathbb{R}, \quad g(w) = \frac{1}{2n} \|w\|^2.$$

The components of h are $h_i(x) = a_i^T x - y_i$, and $F(x) = g(h(x))$.

- The two derivative ingredients are especially simple:

$$J_h(x) = A \in \mathbb{R}^{n \times d}, \quad \nabla g(w) = \frac{w}{n} \in \mathbb{R}^n.$$

$$\begin{aligned} \nabla F(x) &= J_h(x)^T \nabla g(h(x)) \\ &= \frac{1}{n} A^T (Ax - y). \end{aligned}$$

Theorem: Fundamental theorem of calculus

If $\phi : [0, 1] \rightarrow \mathbb{R}$ is continuously differentiable, then

$$\phi(1) - \phi(0) = \int_0^1 \phi'(t) dt.$$

- ▶ How to extend such identity to continuously differentiable $f : \mathbb{R}^d \rightarrow \mathbb{R}$?
- ▶ Let $u, v \in \mathbb{R}^d$. Restrict f to the line segment from u to v :

$$\phi(t) = f(u + t(v - u)), \quad \phi'(t) = \nabla f(u + t(v - u))^T (v - u).$$

- ▶ Applying the above theorem to ϕ gives

$$f(v) - f(u) = \int_0^1 \nabla f(u + t(v - u))^T (v - u) dt.$$

- ▶ Taylor expansions approximate this integral using only derivative at u .

Recall. For $p > 0$, $r(s) = o(\|s\|^p)$ means $r(s)/\|s\|^p \rightarrow 0$ as $s \rightarrow 0$.

Theorem

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and fix $x \in \mathbb{R}^d$.

First-order expansion. If f is differentiable at x , then, as $s \rightarrow 0$,

$$f(x + s) = f(x) + \nabla f(x)^T s + o(\|s\|).$$

Second-order expansion. If f is twice continuously differentiable near x , then, as $s \rightarrow 0$,

$$f(x + s) = f(x) + \nabla f(x)^T s + \frac{1}{2} s^T \nabla^2 f(x) s + o(\|s\|^2).$$

- ▶ Here $\nabla^2 f(x) = [\partial^2 f(x)/(\partial x_i \partial x_j)]_{i,j=1}^d$ is the Hessian.
- ▶ Taylor expansions are central to the design and analysis of many optimization algorithms.